Project Checkpoint #1

Vetrivel Maheswaran
vm6923@rit.edu

I. RECENT PROGRESS

I planned the project as a staged pipeline so each part can be completed, tested, and measured independently. The goal is to build a hybrid RAG system and evaluate whether retrieval and grounding reduce hallucination compared to a vector-only baseline [1], [3].
Dataset (what "data" means here): The dataset is the document corpus used as the knowledge base for the RAG system. It can be a single PDF/DOCX or a collection of domain-specific documents. Each document is converted into text, cleaned, split into chunks, and indexed. The chunks plus metadata (document ID, page/page-range) form the searchable corpus. This structured retrieval setup follows common Retrieval-Augmented Generation design principles [1], [3].

So far, I have completed up to Stage 7.
1. In the Initial Stage, I built a job workflow that tracks progress for each document and each stage. When a user uploads documents, a job is created, and the system records which stage each document is currently in and whether it succeeded or failed. The UI polls job status so progress is visible while the pipeline runs. Fig 1 shows the job workflow interface where each processing stage (ingest, extract, clean, chunk, embed, index) is tracked and displayed to the user during execution.

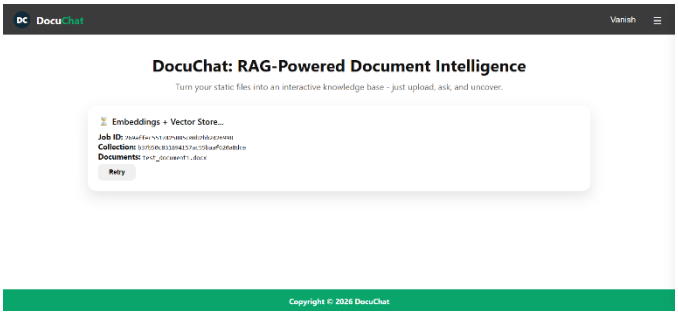

Fig 1. Stage-Based Job Workflow Interface.

2. The upload and document registration flow was then structured. Documents are stored under a collection, and each document gets a deterministic identifier based on the file content. This prevents duplicate ingestion and ensures consistent reruns. Collection metadata tracks associated documents and jobs.

3. Next, I built a document parsing for PDF and DOCX. The output is stored as structured text, keeping organization like page boundaries. The extracted output is stored as an intermediate "processed" representation, so later stages do not need to re-read the raw file. I also added fallback handling, so extraction still produces usable text even when documents contain difficult formatting.

4. I added a cleaning step after extraction to normalize text. This reduces noise from formatting artifacts and makes chunking and embeddings more stable later.

5. Chunking was designed to split documents into meaningful units rather than purely fixed windows. Each chunk retains metadata linking it to its original source location, supporting later grounding and citation. Structured chunking plays a role in hallucination control [3].

6. Embedding generation was incorporated for each chunk, along with caching logic to prevent redundant computation. To evaluate this stage, I recorded efficiency metrics including chunk reuse and execution time. Table 1 shows that during the second upload, one chunk was reused, resulting in a 33.33% avoidance rate and reduced wall time. This confirms that the caching mechanism improves efficiency without affecting indexing consistency.
The embedding avoidance rate is calculated as:

$$\textit{Avoidance Rate (\%)} = (\textit{Skipped Chunks} / \textit{Total Chunks}) \times 100$$

Estimated time saved is calculated as:

$$\textit{Time Saved} = \textit{Skipped Chunks} \times \textit{Average Embedding Time per Chunk}$$

These metrics quantify embedding reuse efficiency and execution improvement during repeated uploads.

Metric	First Upload	Second Upload
Total Documents	1	2
Embedded / Skipped	3 / 0	2 / 1
Avoidance Rate	0%	33.33%
SQLite Hits	0	1
Wall Time (s)	29.728	6.687
Time Saved (s)	0	3.331

Table 1. Embedding Performance Comparison.

7. FAISS-based indexing enables semantic similarity search across all chunk embeddings. The index is stored with metadata mappings to documents and chunk identifiers. Retrieval quality directly influences hallucination behavior in RAG systems [1],

[3]. Future evaluation of hallucination behavior will consider techniques inspired by recent hallucination detection approaches [4], as well as confidence-aware and abstention-based strategies for reliability improvement [5], [6].

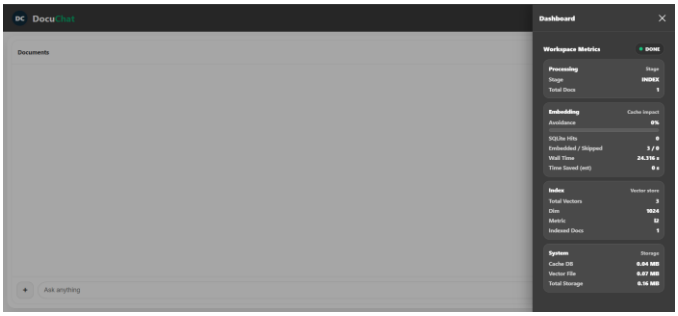

Fig 2. Chat Interface with System Status Panel.

At this stage, the pipeline is completed up to vector indexing. All chunks were embedded and stored successfully, and the FAISS index was built using the expected embedding dimensions. This confirms that the vector backbone is operational. Once embedding and indexing are completed, the system transitions to the chat interface. Fig 2 shows the chat UI along with the system status panel displaying embedding metrics, index statistics, and processing stage information. This verifies that the embedding and indexing stages executed correctly and that the system is ready for retrieval integration.

II. QUESTIONS/CONCERNS

1. The current extraction works well for normal text documents, but it is not fully accurate for scientific equations and complex tables. Some structure may be lost during parsing. I plan to improve this in the next checkpoint and try better handling for such elements.

III. NEXT STEPS

Now, after successfully building the FAISS index for the vector embeddings, I am working on the following stages:

8. Build the graph layer using Neo4j to represent relationships between entities or document segments extracted from the corpus, similar to recent hybrid vector–graph RAG approaches [2], [7].
9. Design and build hybrid retrieval by combining vector-based similarity search with graph-based reasoning as explored in hybrid RAG architectures [2], [7]. Then integrate grounded generation where responses reference specific supporting chunks.
10. Add conversation history storage so the system can maintain context across multiple user interactions.

These are my current plans to work on before the next checkpoint.

REFERENCES

[1] K. Sawarkar, A. Mangal and S. R. Solanki, "Blended RAG: Improving RAG (Retriever-Augmented Generation) Accuracy with Semantic Search and Hybrid Query-Based Retrievers," 2024 IEEE 7th International Conference on Multimedia Information Processing and Retrieval (MIPR), San Jose, CA, USA, 2024, pp. 155-161, doi: 10.1109/MIPR62202.2024.00031.

[2] Sarmah, Bhaskarjit, Benika Hall, Rohan Rao, Sunil Patel, Stefano Pasquali, and Dhagash Mehta. “HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation for Efficient Information Extraction.” arXiv:2408.04948. Preprint, arXiv, August 9, 2024. https://doi.org/10.48550/arXiv.2408.04948.

[3] R. Ko÷, M. K. Gürkan and F. T. Yarman Vural, "ReRag: A New Architecture for Reducing the Hallucination by Retrieval-Augmented Generation," 2024 9th International Conference on Computer Science and Engineering (UBMK), Antalya, Turkiye, 2024, pp. 961-965, doi: 10.1109/UBMK63289.2024.10773428.

[4] Hu, Haichuan, Congqing He, Xiaochen Xie, and Quanjun Zhang. “LRP4RAG: Detecting Hallucinations in Retrieval-Augmented Generation via Layer-Wise Relevance Propagation.” arXiv:2408.15533. Preprint, arXiv, June 27, 2025. https://doi.org/10.48550/arXiv.2408.15533.

[5] Y. A. Yadkori, I. Kuzborskij, D. Stutz, et al., “Mitigating LLM Hallucinations via Conformal Abstention,” arXiv:2405.01563, Apr. 4 2024. doi: 10.48550/arXiv.2405.01563.

[6] W. T. Chan, K. Hung, R. Ho, and G. M.-T. Man, “Optimizing Confidence Scoring in RAG-Based LLM Chatbots for Technical Support Services: A Prompt Engineering Approach,” 2025 IEEE 15th Symposium on Computer Applications & Industrial Electronics (ISCAIE), May 2025, pp. 540–545. doi: 10.1109/ISCAIE64985.2025.11081158

[7] S. Knollmeyer, M. U. Akmal, L. Koval, S. Asif, S. G. Mathias and D. Groβmann, "Document Knowledge Graph to Enhance Question Answering with Retrieval Augmented Generation," 2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA), Padova, Italy, 2024, pp. 1-4, doi: 10.1109/ETFA61755.2024.10711054.